

Project description

This project implements and benchmarks three CUDA kernels for causal self-attention on a single attention head: a naive baseline, a shared-memory tiled kernel, and an online-softmax tiled kernel. Its purpose is to show how kernel structure on the GPU changes memory usage and runtime performance while preserving numerical correctness, using the same attention operation expressed in three designs. In the naive and shared-memory tiled implementations, the code explicitly forms the attention score matrix and then applies softmax and value weighting, whereas the online-softmax tiled implementation follows the FlashAttention idea of processing one key/value tile at a time and updating the softmax normalization incrementally so the full score matrix is never stored in global memory. The project draws on ThunderKittens for a tile-oriented view of AI kernels and on online-softmax formulations used to derive FlashAttention, adapting those concepts to a plain-CUDA setting suitable for a teaching benchmark.

A central goal is to understand why the kernels behave differently, not just to demonstrate that they run. To support that goal, the repository includes a CPU reference implementation for validation, timing with CUDA events, and experiments across multiple sequence lengths and head dimensions, so both numerical error and runtime can be examined side by side.

Functions

`cpu_attention`

Purpose: Compute a correct reference result on the CPU for one batchless causal attention problem, so every CUDA kernel can be checked for numerical accuracy.

Pseudocode:

```
function cpu_attention(Q, K, V, O, n, d):
    scale = 1 / sqrt(d)
    for each row i in 0..n-1:
        m = -infinity
        for each key j in 0..i:
            score[j] = dot(Q[i], K[j]) * scale
            m = max(m, score[j])
        z = 0
        for each key j in 0..i:
            score[j] = exp(score[j] - m)
            z += score[j]
        for each output coordinate x in 0..d-1:
            acc = 0
            for each key j in 0..i:
```

```
acc += (score[j] / z) * V[j][x]
O[i][x] = acc
```

What it does:

- Computes causal attention row by row.
- Applies a numerically stable softmax using the row maximum.
- Produces the ground-truth output for validation.

scores_naive

Purpose: Compute the full lower-triangular score matrix $S = QK^T/\sqrt{d}$ on the GPU.

Pseudocode:

```
kernel scores_naive(Q, K, S, n, d, scale):
    i = row index from block and thread
    j = column index from block and thread
    if i or j out of range:
        return
    if j > i:
        S[i][j] = -infinity
        return
    dot = 0
    for x in 0..d-1:
        dot += Q[i][x] * K[j][x]
    S[i][j] = dot * scale
```

What it does:

- Launches a 2D grid over the score matrix.
- Computes one score entry per thread.
- Enforces the causal mask by writing negative infinity above the diagonal.

row_softmax

Purpose: Convert each row of the score matrix into probabilities.

Pseudocode:

```
kernel row_softmax(S, n):
    i = block index, so one block handles one row
    local_max = maximum score seen by this thread over strided columns
    reduce local_max across the block to get row max m
    local_sum = sum of exp(S[i][j] - m) over strided columns
    overwrite S[i][j] with exp(S[i][j] - m)
```

```

    reduce local_sum across the block to get row denominator z
    for each strided column j:
        S[i][j] = S[i][j] / z

```

What it does:

- Uses block-level reduction in shared memory.
- Computes a stable softmax in two passes over the row.
- Reuses the score matrix buffer to store probabilities.

weighted_value

Purpose: Compute $O = PV$, where P is the row-softmax output.

Pseudocode:

```

kernel weighted_value(P, V, O, n, d):
    i = output row
    x = output coordinate
    if out of range:
        return
    acc = 0
    for j in 0..i:
        acc += P[i][j] * V[j][x]
    O[i][x] = acc

```

What it does:

- Produces the final attention output after softmax.
- Uses the causal structure, so only keys up to row i contribute.

launch_naive_attention

Purpose: Run the full naive GPU pipeline.

Pseudocode:

```

function launch_naive_attention(Q, K, V, S, O, n, d, scale):
    launch scores_naive
    launch row_softmax
    launch weighted_value

```

What it does:

- Uses three separate kernels.
- Materializes the entire score matrix in global memory.

- Is simple and easy to reason about, but memory-heavy.

scores_tiled

Purpose: Compute the same score matrix as the naive version, but stage tiles of Q and K in shared memory.

Pseudocode:

```
kernel scores_tiled(Q, K, S, n, d, scale):
    load one TILE-by-d tile of Q into shared memory
    load one TILE-by-d tile of K into shared memory
    synchronize
    qi = global query row
    kj = global key row
    if out of range:
        return
    if kj > qi:
        S[qi][kj] = -infinity
        return
    dot = 0
    for x in 0..d-1:
        dot += shared_Q[local_q][x] * shared_K[local_k][x]
    S[qi][kj] = dot * scale
```

What it does:

- Reuses Q and K values from shared memory instead of repeatedly reading them from global memory.
- Demonstrates the basic GPU optimization idea of tiling.
- Still stores the full score matrix globally, so it does not solve the memory-footprint issue.

row_softmax_tiled

Purpose: Same as `row_softmax`, but kept as a separate function for the tiled pipeline.

Pseudocode:

```
kernel row_softmax_tiled(S, n):
    same logic as row_softmax
```

What it does:

- Performs the same stable softmax reduction.
- Keeps the tiled pipeline structurally separate.

weighted_value_tiled

Purpose: Same as `weighted_value`, but used by the tiled pipeline.

Pseudocode:

```
kernel weighted_value_tiled(P, V, O, n, d):  
    same logic as weighted_value
```

What it does:

- Produces the final output for the tiled score pipeline.

launch_tiled_attention

Purpose: Run the score-materializing tiled GPU pipeline.

Pseudocode:

```
function launch_tiled_attention(Q, K, V, S, O, n, d, scale):  
    launch scores_tiled  
    launch row_softmax_tiled  
    launch weighted_value_tiled
```

What it does:

- Preserves the same overall algorithm as the naive implementation.
- Changes only the score stage to use shared-memory tiling.
- Helps isolate the effect of tiling from the effect of fusion.

attention_online_tiled_kernel

Purpose: Compute causal attention without storing the full score matrix, using the online-softmax recurrence tile by tile.

Pseudocode:

```
kernel attention_online_tiled_kernel(Q, K, V, O, n, d, scale):  
    row = one query row handled by one block  
    load Q[row] into shared memory  
    initialize:  
        old_m = -infinity  
        old_l = 0  
        out_acc = zero vector of length d  
    for tile_start in 0, TILE, 2*TILE, ... up to row:  
        load K tile and V tile into shared memory  
        synchronize
```

```

    for each key in the tile:
        score[j] = dot(Q[row], K_tile[j]) * scale
    tile_max = max(score[j] over tile)
    new_m = max(old_m, tile_max)
    tile_sum = 0
    for each key in the tile:
        weight[j] = exp(score[j] - new_m)
        tile_sum += weight[j]
    new_l = old_l * exp(old_m - new_m) + tile_sum
    for each output coordinate x:
        pv = sum_j weight[j] * V_tile[j][x]
        out_acc[x] =
            (old_l * exp(old_m - new_m) / new_l) * out_acc[x]
            + pv / new_l
    old_m = new_m
    old_l = new_l
    write out_acc to O[row]

```

What it does:

- Processes one query row at a time.
- Loads only one tile of keys and values at a time into shared memory.
- Maintains the running max and running denominator needed for stable online softmax.
- Updates the output incrementally, so there is never a need to write or read a full $n \times n$ score matrix.

launch_online_tiled_attention

Purpose: Run the full online-softmax tiled kernel.

Pseudocode:

```

function launch_online_tiled_attention(Q, K, V, O, n, d, scale):
    launch attention_online_tiled_kernel with one block per query row

```

What it does:

- Uses a single fused kernel instead of a three-stage pipeline.
- Reduces global-memory traffic by avoiding the explicit score matrix.
- Captures the main algorithmic idea behind FlashAttention-style attention kernels.²

main

Purpose: Generate data, run the kernels, compare against the CPU reference, and report timing and error.

Pseudocode:

```
function main(args):
    parse n, d, iters, mode
    validate input sizes
    allocate host arrays Q, K, V, O_ref, O_gpu
    fill Q, K, V with random values
    allocate device arrays
    copy Q, K, V to device
    compute O_ref on CPU
    define report(name, kernel_launch):
        do one warmup launch
        synchronize
        start timer
        repeat kernel_launch iters times
        stop timer
        copy O_gpu back to host
        compute max absolute error against O_ref
        print timing and error
    if mode includes naive:
        report(naive, launch_naive_attention)
    if mode includes tiled:
        report(shared_tiled, launch_tiled_attention)
    if mode includes online:
        report(online_tiled, launch_online_tiled_attention)
    free device memory
```

What it does:

- Makes the project reproducible.
- Ensures every GPU result is checked for correctness.
- Reports average kernel time, which makes comparisons cleaner than a single noisy measurement.

Control flow

The program starts in `main`, where it parses command-line arguments, allocates host and device memory, generates random input matrices Q , K , and V , and computes a CPU reference output. It then selects one or more GPU attention implementations based on the requested mode and runs them against the same inputs. In the naive pipeline, three kernels run in sequence: score computation, row-wise softmax, and weighted value accumulation. The shared-memory tiled pipeline follows the same three-stage structure, but improves the score computation by caching tiles of Q and K in shared memory instead of repeatedly loading them from global memory. The online-softmax tiled pipeline uses a single fused kernel, where each

block owns one query row and iterates over key–value tiles while maintaining a running softmax state and output accumulator until the full causal row has been processed. After each GPU run, the result is copied back to the CPU, compared with the reference output, and summarized using the average runtime and maximum absolute error.

Lessons learned

This project made clear that correctness and performance engineering are related but distinct goals. The naive kernel was the easiest to understand and verify, because each stage of the attention computation was isolated in its own kernel with a straightforward control flow. The shared-memory tiled kernel introduced a central GPU optimization idea: reusing data through on-chip memory instead of repeatedly loading Q and K from global memory, which reduces bandwidth pressure even though it still materializes the full score matrix and therefore does not solve the attention memory-footprint bottleneck. The online-softmax tiled kernel was the most conceptually demanding component, because it required maintaining a stable softmax normalization incrementally across tiles while simultaneously updating the output vector in place. Even when this educational implementation was not always faster on small test sizes, it embodied the key algorithmic improvement by reducing global-memory traffic and aligning with the structure used in modern fused attention kernels. Overall, the main challenge was not just writing CUDA code, but reasoning carefully about data layout, synchronization, causal masking, numerical stability, and how to translate a mathematical attention formula into a tiled GPU algorithm that preserves correctness while exposing performance tradeoffs.

Significance

This project moves past a basic CUDA demonstration by implementing and validating several kernel designs for the same nontrivial linear-algebra workload, measuring their behavior empirically, and including a fused attention kernel based on online softmax rather than a superficial parallelization of the standard score-and-softmax steps.